

Canonical Huffman file compressor

Benjamin Jones | 41052 Advanced Algorithms | Programming Assignment 1 | Track B

What I built

The artifact is a lossless command-line file compressor based on canonical Huffman coding. It compresses arbitrary bytes into a self-contained HUF1 archive, reconstructs the original file, and exposes an inspection command. The algorithm is implemented in Python rather than delegated to a compression library. Python's `heapq` supplies the priority queue; `Counter` counts symbols; `struct` stores binary metadata; `zlib` is used only for CRC32 integrity checking. The separate comparison uses Python's `gzip` library as a baseline.

Huffman coding was selected from the approved Track B topic list. The tool goes beyond an isolated tree implementation: it includes a persistent binary format, a decoder, validation of malformed archives, overwrite protection, a reproducible demonstration, integration tests, and measured comparison with `gzip`. It is intended as an understandable small-file educational compressor, not a production archive format.

Representation and code organisation

The alphabet is byte values 0 through 255. This avoids conflating characters with bytes and works for UTF-8 text or binary input. `code_lengths` constructs the Huffman tree; `canonical_codes` assigns deterministic codewords; `compress` packs them; `inspect_archive` validates the format; `decompress` reconstructs a decoding trie; `main` implements the command-line interface. Tests and benchmarking are separate files.

Each priority-queue entry has a frequency, a unique serial number and a tree. A leaf is an integer byte value; an internal tree is a pair of children. The serial number handles equal frequencies without comparing incompatible tree types and gives deterministic tie handling [1]. Leaf depths provide lengths. The singleton case is explicitly assigned length one; an empty input has no codes.

Algorithm and correctness

For each symbol s , count frequency $f(s)$. Insert its leaf into a min-heap. While two or more trees remain, remove the two smallest weights and replace them with a parent of their combined weight. Huffman's greedy step minimises weighted leaf depth among binary prefix codes: two least-frequent symbols can be placed as deepest siblings in an optimal tree; contracting them reduces the problem to a smaller alphabet. Repeating the contraction yields the tree constructed by the algorithm.

The construction invariant is that heap trees partition the observed alphabet and each tree weight equals the sum of its leaf frequencies. It holds initially because every symbol has one leaf. A merge removes two disjoint trees and reinserts their union, preserving both properties. At termination, the remaining tree contains every symbol exactly once. Tree leaf paths are prefix-free because a leaf cannot also be an ancestor of another leaf.

Canonical assignment sorts symbols by (length, byte value). Starting from code zero, it shifts to each new length, assigns the code, and increments. Code lengths preserve the Huffman payload cost while the canonical ordering makes the encoding reproducible from lengths alone. The decoder rejects oversubscribed lengths, incomplete multi-symbol tables, and invalid singleton lengths rather than trusting arbitrary metadata.

The payload is packed most-significant bit first. An accumulator holds unflushed bits; complete bytes are emitted; the remaining bits are shifted into the high positions of the last byte. The decoder uses exactly the recorded number of valid bits, excluding zero padding. After reaching a trie leaf, it emits the symbol and resets to the root. Removing that reset breaks the next code's traversal. Successful decoding requires ending at the root, producing the declared byte count, and passing CRC32.

For n input bytes, k observed symbols and B encoded bits, frequency counting is $O(n)$, heap construction and merging $O(k \log k)$, and canonical sorting $O(k \log k)$. Packing and bit-by-bit decoding are $O(B)$; k is at most 256 and Huffman depth is at most 255. The whole-file design requires $O(n + B/8 + k)$ memory when input/output buffers are

included. It deliberately does not stream large files.

The tool and its interface

The CLI has `compress INPUT OUTPUT`, `decompress INPUT OUTPUT`, and `inspect INPUT` subcommands. File names are explicit; `--force` is required to overwrite an output. Input and output resolving to the same path are rejected. Exclusive output creation also prevents an existence-check race. Failed format validation returns a readable error and exit status 2. Decompression checks a default 256 MiB output cap before constructing output, with an explicit `--max-output` override.

The HUF1 archive begins with a 26-byte big-endian header: 4-byte magic, 8-byte original size, 8-byte payload bit count, 4-byte CRC32 and 2-byte symbol count. Each table entry adds a byte value and one-byte code length. The decoder validates symbol count, duplicate symbols, lengths, payload size, zero padding, decoded size and checksum. CRC32 is an accidental-corruption check, not cryptographic authentication. These checks improve usability but do not constitute an adversarial security audit.

Worked example

Running `python demo.py` compresses `examples/message.txt`, inspects the archive and restores it. The measured sample contains 530 bytes and produces a 362-byte archive, a total archive/input ratio of 0.683. There are 24 observed symbols and 2,300 meaningful payload bits. The original and restored SHA256 digests are identical, and an explicit byte comparison passes. A checksum-modified archive is rejected with a CRC32 integrity error. The demonstration reruns safely within `demo-output` and does not change the original input.

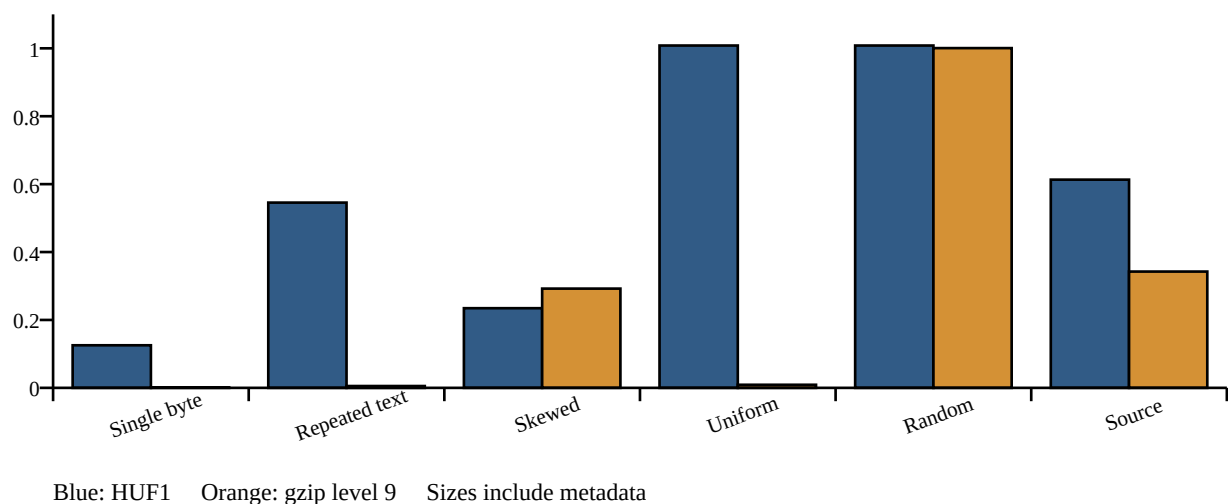
Validation and comparison

Nine automated tests pass. Coverage includes empty input, singleton input, all 256 bytes, Unicode represented as UTF-8, seeded random and skewed input, deterministic output, prefix freedom, a known optimal weighted cost, and exhaustive merge-order optimality checks for small alphabets. Negative cases include every truncation position of a sample archive, trailing bytes, duplicate symbols, zero lengths, invalid table capacity, nonzero padding, payload/checksum/size corruption and output limits. The CLI integration test verifies actual subprocesses, files, round trips and no-overwrite behaviour. These are evidence for the tested cases, not a formal proof covering all possible archives.

The comparison uses eight datasets and five repetitions per codec. Six datasets are small controls or synthetic byte sources; a repeated sentence represents highly repetitive text; the implementation source supplies a real project file. Larger synthetic inputs have 65,536 bytes. Random generation uses seed 41052. Timing is the median in-memory encode/decode wall time, excluding disk I/O; gzip uses level 9 and mtime 0 [2]. Each timed result must round-trip exactly. Full timings, ratios and environment are in `results/benchmark.csv` and `results/environment.json`.

Dataset	Bytes in	HUF bytes	gzip bytes	HUF encode ms	gzip encode ms
empty	0	26	20	0.005	0.036
tiny text	13	54	33	0.025	0.037
single symbol	65536	8220	97	9.766	0.23
repeated text	65536	35754	351	11.912	0.388
skewed symbols	65536	15373	19155	11.317	71.296
uniform bytes	65536	66074	597	10.971	0.274
random bytes	65536	66074	65574	11.191	1.131
source code	8435	5172	2889	1.473	0.196

Total archive size / input size (lower is better)



Interpretation

Archive sizes include headers and symbol tables. Single-symbol input requires one bit per byte here, plus 28 metadata bytes, yielding 8,220 bytes. A zero-bit singleton representation could reduce that case substantially, but the one-bit design keeps encoding and validation uniform. Tiny text expands because the table and header outweigh the saved payload bits. Uniform and random 256-symbol inputs produce 66,074-byte archives: 65,536 payload bytes plus 538 metadata bytes. Frequency coding cannot eliminate information in an approximately uniform byte distribution.

On the skewed-symbol dataset, HUF1 produces 15,373 bytes against gzip's 19,155. This result shows one workload where the simple global frequency model is effective; it does not establish broad superiority. Conversely, repeated text becomes 35,754 bytes with HUF1 but 351 with gzip, and the repeated 0-to-255 pattern becomes 66,074 versus 597. The latter has uniform byte frequencies yet strong sequence repetition. This distinguishes byte-frequency skew from repeated substrings. DEFLATE, used by gzip, combines LZ77 repeated-sequence references with Huffman coding [3]; this tool has no dictionary model.

Timing differences also reflect engineering, not just asymptotic algorithm choices. The custom encoder/decoder execute Python loops, while gzip invokes a mature native implementation. Gzip is faster on most measured inputs,

but the level-9 skewed-symbol encoding is slower in this run. Five local measurements on mostly synthetic 64 KiB data cannot support claims about all file types, machines or compression levels. Future work should include independent corpora, multiple sizes, block-based streaming, peak memory, and a stored/raw fallback when compression expands data.

What the project revealed

The most useful result is that equal symbol frequencies do not imply the absence of compressible structure. The `uniform_bytes` dataset contains each byte equally often, so single-byte Huffman coding uses eight-bit codes and adds metadata. Gzip still compresses the ordered repetition to 597 bytes. This explains why optimality within the class of symbol prefix codes is a narrower claim than being the best file compressor.

A second finding concerns persistence. A correct in-memory tree is not enough for a reliable tool: the decoder needs an unambiguous code table and a precise endpoint for the bitstream. Original length, payload bit count, canonical lengths and padding validation all solve different parts of that problem. The round-trip tests alone would not have demonstrated malformed-input handling or optimal payload cost, so the suite includes independent small-case cost checks and negative archive cases.

AI use

I used OpenAI Codex as a development assistant for algorithm research and explanation, implementation, testing, benchmarking, documentation, and report preparation. The assistance was substantial: it produced the initial compressor and CLI implementation, test suite, benchmark scripts, and report draft, and ran validation in the development workspace. I directed the project requirements and prepared the narrated demonstration myself. The submitted work therefore combines my project direction and presentation with AI-assisted code and writing; AI assistance was not limited to spelling or minor suggestions.

Two specific corrections arose during development. First, the initial encoder computed its bit count with `Counter(data)` inside the per-symbol comprehension. That rescanned the entire input once for each symbol even though one frequency count sufficed. The generated code was corrected to construct the counter once and reuse it. The original version round-tripped correctly, but its unnecessary repeated work would distort encoding timings and weaken the implementation-quality claim. Final benchmark measurements were taken after that correction.

Second, the generated CLI test initially used `tempfile.TemporaryDirectory`. In this Windows runtime its restrictive directory creation produced `PermissionError` when writing test files. Moving it inside the project did not solve the error. The test was changed to create a uniquely named ordinary project directory, inherit usable permissions, and clean up only its own files. The initial test run had eight passing tests and one error; the final run has all nine passing. This was an environmental assumption in AI-generated testing code, rather than a Huffman algorithm defect.

The code walkthrough explains the frequency-counting and merge process, canonical reconstruction, bit packing, and the decoder's reset to the root. Confidence in the tool's behaviour is supported by round-trip checks, matching hashes, malformed-input tests, and an independent small-case optimality check, rather than relying on generated explanations alone. These checks do not establish a complete formal proof, hostile-input security, or large-file performance; those areas remain outside the verified scope. The recorded development checks include assistant-executed tests and are not presented as independent student testing.

References

- [1] Python documentation, `heapq` priority queue implementation notes. <https://docs.python.org/3/library/heapq.html>
- [2] Python documentation, `gzip` compression interface. <https://docs.python.org/3/library/gzip.html>
- [3] P. Deutsch, DEFLATE Compressed Data Format Specification version 1.3, RFC 1951, 1996. <https://www.rfc-editor.org/rfc/rfc1951>

[4] 41052 Programming Assignment 1 brief and AI policy.
<https://edstem.org/au/courses/37005/lessons/115269/slides/792282>

[5] Approved topic list, Huffman coding Track B.
<https://edstem.org/au/courses/37005/lessons/111579/slides/768994>